Lab 2 – From LeNet to AlexNet

Group 22

Madison O’Brien 20326252

Rachel Millar 20320587

ELEC 475, Fall 2026

2. Architectural specifications

All models intake a 3x64x64 RGB input. Spatial sizes of the inputs follow N_out = [(N_in – K +2p)/s] +1. The receptive fields (RF) follow the equation RF_out = RF_in + (K-1) x j_in, where j_in the cumulative stride, and j_out = j_in x s.

Base LeNet-5

Layer	Configuration	Output	Parameters	RF
Conv1 + Tanh	5x5, s=1, p=0, 6 filters	6x60x60	456	5
AvgPool1	2x2, s=2	6x30x30	0	6
Conv2 + Tanh	5x5, s=1, p=0, 16 filters	16x26x26	2,416	14
AvgPool2	2x2, s=2	16x13x13	0	16
FC1 + Tanh	2,704 → 120	120	324,600	–
FC2 + Tanh	120 → 84	84	10,164	–
FC3	84 → 10	10	850	–

The flattened feature vector has 16 x13 x13 = 2,704 elements. In total this is 338,486 parameters, 3.61 M MACs, 7.22 MFLOPs (profiler). Over 95% of the parameters sit in FC1.

Modernized LeNet

This version replaces Tanh with ReLU and average pooling with overlapping max pooling (3x3, stride 2). The convolutions use padding 2 so that the pooling stage works on 64x64 maps.

Layer	Configuration	Output	Parameters	RF
Conv1 + ReLU	5x5, s=1, p=2, 6 filters	6x64x64	456	5
MaxPool1	3x3, s=2	6x31x31	0	7
Conv2 + ReLU	5x5, s=1, p=2, 16 filters	16x31x31	2,416	15
MaxPool2	3x3, s=2	16x15x15	0	19
FC1 + ReLU	3,600 → 120	120	432,120	–

FC2 + ReLU	120 → 84	84	10,164	–
FC3	84 → 10	10	850	–

The flattened vector grows to 16x15x15 = 3,600 elements. In total this is 446,006 parameters, 4.68 M MACs, 9.35 MFLOPs. The extra 107,520 parameters are almost entirely within FC1.

AlexNet-64

AlexNet-64 has five convolutional stages and a dropout-regularized classifier. Normalization is configurable (none, LRN or BatchNorm).

Layer	**Configuration**	**Output**	**Parameters**	**RF**
Conv1	5x5, s=2, p=2, 64	64x32x32	4,864	5
MaxPool1	3x3, s=2	64x15x15	0	9
Conv2	5x5, s=1, p=2, 192	192x15x15	307,392	25
MaxPool2	3x3, s=2	192x7x7	0	33
Conv3	3x3, s=1, p=1, 384	384x7x7	663,936	49
Conv4	3x3, s=1, p=1, 384	384x7x7	1,327,488	65
Conv5	3x3, s=1, p=1, 256	256x7x7	884,992	81
MaxPool3	3x3, s=2	256x3x3	0	97
Dropout(0.5) + FC1 + ReLU	2,304 → 1,024	1,024	2,360,320	–
Dropout(0.5) + FC2 + ReLU	1,024 → 1,024	1,024	1,049,600	–
FC3	1,024 → 10	10	10,250	–

In total there are 6,608,842 parameters without normalization and 6,611,402 with BatchNorm (BN adds 2,560 learnable scale/shift parameters and its running statistics are not trainable). The profiler reports 218.35 M MACs (436.70 MFLOPs) without normalization and 218.98 M MACs (437.97 MFLOPs) with BatchNorm. By Conv5 and MaxPool3 the receptive field (81 and 97 pixels) exceeds the 64-pixel input, so the final features see the whole image.

For BatchNorm, the layer sits between convolution and ReLU in all five stages. For LRN (size 5, α=1e-4, β=0.75, k=2), it sits after ReLU and before pooling in stages 1 and 2 only, following the original 2012 design.

The lab instructions indicate about 6.3 M parameters and 250–257 MFLOPs for AlexNet-64, and about 4.8 and 8.2 MFLOPs for the two LeNets. The found measurements (6.61 M parameters, 437 MFLOPs, 7.22 and 9.35 MFLOPs) differ from these values. The parameter count that was outputted matches a hand calculation from the specified layer sizes. The profiler counts one MAC as two FLOPs, so the manual's values appear to use a different counting convention or approximation. The profiler's measured values are reported throughout.

PyTorch Zoo AlexNet baseline

Commented [MO1]: Maddy

Loads the official torchvison AlexNet and it adapts it for the target dataset class counts. For CIFAR-10 its output is changed from 1000 to 10 and for Tiny ImageNet it changed from 1000 to 200.

Layer	Configuration	Output	Parameters	RF
Conv1 + ReLU	11x11 s=4 p=2 64	64x15x15	23296	11
MaxPool1	3x3 s=2	64x7x7	0	19
Conv2 + ReLU	5x5 s=1 p=2 192	192x7x7	307392	51
MaxPool2	3x3 s=2	192x3x3	0	67
Conv3 + ReLU	3x3 s=1 p=1 384	384x3x3	663936	99
Conv4 + ReLU	3x3 s=1 p=1 256	256x3x3	884992	131
Conv5 + ReLU	3x3 s=1 p=1 256	256x3x3	590080	163
MaxPool3	3x3 s=2	256x1x1	0	195
AdaptiveAvgPool2d	output 6x6	256x6x6	0	–
Dropout(0.5) + FC1 + ReLU	9216 → 4096	4096	37752832	–
Dropout(0.5) + FC2 + ReLU	4096 → 4096	4096	16781312	–
FC3	4096 → 10	10	40970	–

3. Empirical results and figures

Unless stated otherwise, runs use CIFAR-10, Adam (lr = 1e-3), batch size 128 and seed 42. AlexNet runs were done on a Tesla T4 GPU. The in-lab LeNet runs used a CPU. The augmentation pipeline was not applied in any run except the *with augmentation* run below, so the *None*entries in Table 7.3a are accurate from this.

LeNet baselines (in-lab, 5 epochs, CPU)

Model	**Val loss**	**Val acc (%)**	**Val Macro F1 (%)**	**Mean epoch time**
Base LeNet-5	1.3148	53.14	53.12	about 21.6 s
Modernized LeNet	1.0064	64.20	63.76	about 21.6 s

Normalization study (AlexNet-64, 10 epochs)

Variant	**Parameters**	**FLOPs**	**Val loss**	**Val acc (%)**	**Val Macro F1 (%)**	**Train acc (%)**	**Δ (train − val)**	**Mean epoch (s)**
None	6,608,842	436.70 M	0.7717	73.52	73.48	74.84	1.32	23.7
LRN	6,608,842	436.70 M	0.7452	75.15	75.13	78.72	3.57	25.5
BatchNorm	6,611,402	437.97 M	0.6172	79.93	80.06	84.86	4.93	23.5

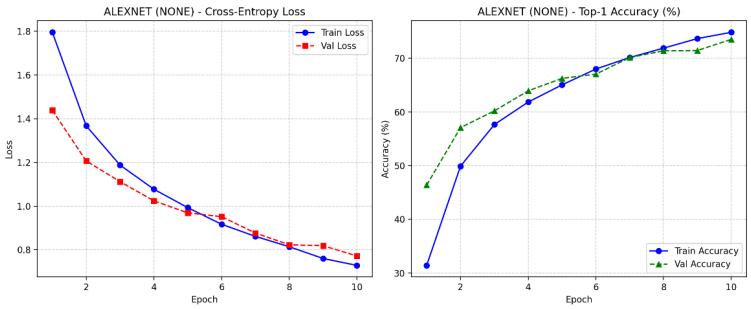


Figure 1: AlexNet, 10 epochs

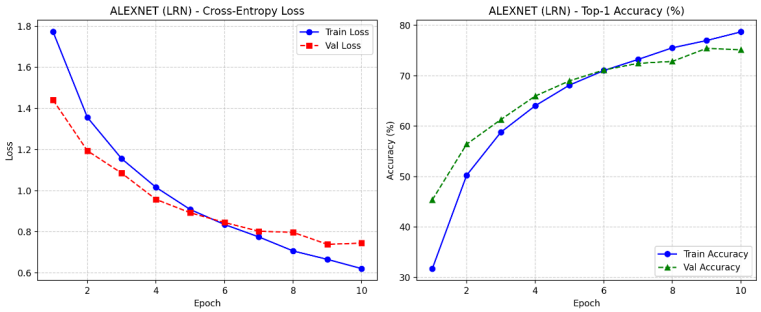


Figure 2: AlexNet + LRN, 10 Epochs

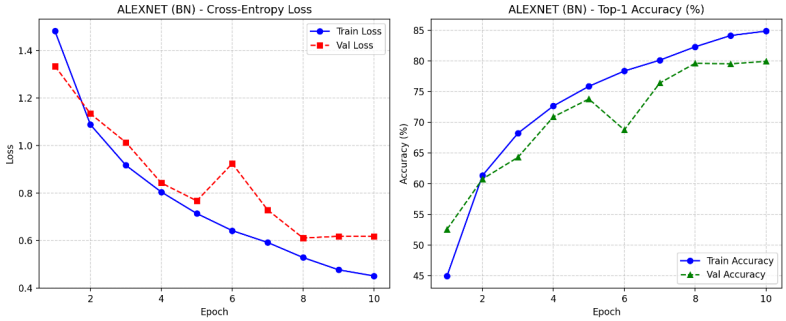


Figure 3: AlexNet + BN, 10 Epochs

Data augmentation (AlexNet-64 + BN, 15 epochs)

Run	Train acc (%)	Val acc (%)	Δ	Val Macro F1 (%)	Val loss	Best val acc (%)	Mean epoch (s)
No augmentation	90.31	78.59	11.72	78.33	0.7104	81.18	24.2
With augmentation	84.64	81.65	2.99	81.26	0.5222	82.83	53.9

Augmentation was RandomCrop (64, padding 4, reflect), RandomHorizontalFlip (p=0.5) and ColorJitter (brightness, contrast and saturation 0.2).

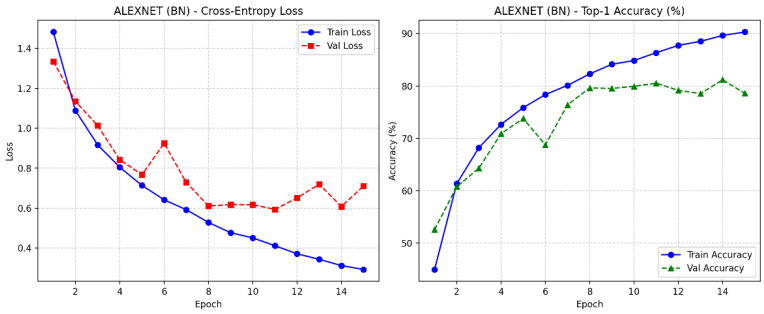


Figure 4: AlexNet + BN, No Augmentations, 15 Epochs

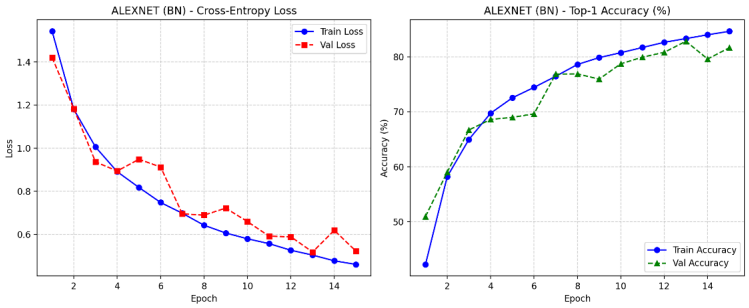


Figure 5: AlexNet + BN, Augmentation, 15 Epochs

Tiny ImageNet benchmark (AlexNet-64 + BN, 10 epochs)

Validation split	Train acc (%)	Val acc (%)	Val Macro F1 (%)	Val loss	Δ (train − val)	Mean epoch (s)
Held-out val/ (best, epoch 9)	25.58	29.20	27.61	3.0554	−3.62	109.3
Held-out val/ (final, epoch 10)	27.14	28.31	26.99	3.0779	−1.17	109.3

PyTorch Zoo AlexNet results

Model	Params (M)	Train acc (%)	Val acc (%)	Val Macro F1 (%)	Val loss	Δ (train − val)	Mean epoch (s)

PyTorch Zoo AlexNet	57.045	67.50	65.24	65.05	1.0138	2.26	24.9

Master ablation tables

Architectural Configuration & Computational Complexity Profile

Model	Conv depth	Normalization & regularization	Augmentation	Params (M)	FLOPs (M)	Mean epoch (s)	Latency (ms)
1. Base LeNet-5	2	None	None	0.338	7.17	21.6 (CPU)	0.35
2. Modernized LeNet	2	None (ReLU + overlapping MaxPool)	None	0.446	9.19	21.6 (CPU)	0.35
3. AlexNet (No Norm)	5	Dropout (p=0.5)	None	6.609	436.70	23.7	1.89
4. AlexNet + LRN	5	LRN + Dropout (p=0.5)	None	6.609	436.70	25.5	1.67
5. AlexNet + BatchNorm	5	BatchNorm2d + Dropout (p=0.5)	None	6.611	437.97	23.5	1.48
6. AlexNet + BN + Aug	5	BatchNorm2d + Dropout (p=0.5)	Pure PyTorch Aug	6.611	437.97	53.9	1.48
7. PyTorch Zoo Alexnet	5	PyTorch Zoo (AdaptiveAvgPool2d)	None	57.045	188.20	24.9	1.91

Empirical Predictive Performance Across Datasets

Model	CIFAR-10 val acc (%)	CIFAR-10 val Macro F1 (%)	Tiny ImageNet val acc (%)	Tiny ImageNet val Macro F1 (%)
1. Base LeNet-5	53.14	53.12	12.41	10.80
2. Modernized LeNet	64.20	63.76	20.48	18.91
3. AlexNet (No Norm)	73.52	73.48	16.46	13.68
4. AlexNet + LRN	75.15	75.13	0.50	0.00
5. AlexNet + BatchNorm	79.93	80.06	30.15	28.25
6. AlexNet + BN + Aug	81.65	81.26	28.31	26.99
7. PyTorch Zoo AlexNet	65.24	65.05	11.03	8.38

Model complexity benchmark (compare_models.py)

Model architecture	Params (M)	MACs (M)	FLOPs (M)	Latency (ms)	Throughput (img/s)
1. LeNet-5 (Base)	0.338	3.59	7.17	0.35	2897.5
2. LeNet-Modern (ReLU+Pool)	0.446	4.59	9.19	0.35	2861.4
3. AlexNet-64 (None)	6.609	218.35	436.70	1.89	527.9
4. AlexNet-64 (LRN)	6.609	218.35	436.70	1.67	598.1
5. AlexNet-64 (BatchNorm)	6.611	218.98	437.97	1.48	674.8
6. Torchvision Zoo AlexNet	57.045	94.10	188.20	1.91	523.3

Extended ablation exploration

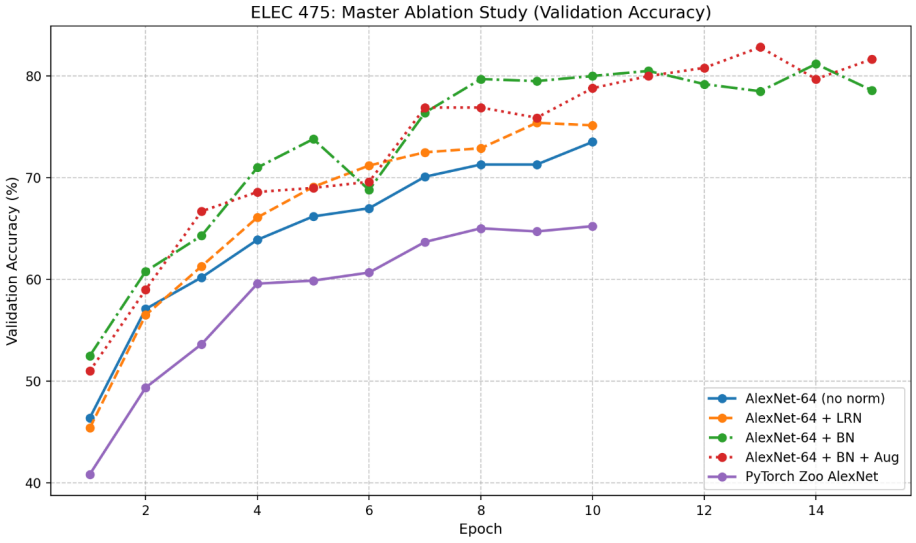
Experiment	Val acc (%)	Δ acc	Val Macro F1 (%)	ΔF1	Train acc (%)	Train − val gap	Mean epoch (s)
Baseline (lr 1e-3; batch 128)	79.93	—	80.06	—	84.86	4.93	23.5
Learning rate 3e-4	81.02	+1.09	81.06	+1.00	89.07	8.05	23.1
Batch size 64	79.17	−0.76	79.17	−0.89	84.03	4.86	25.0

Multi-seed results (Mean ± Std)

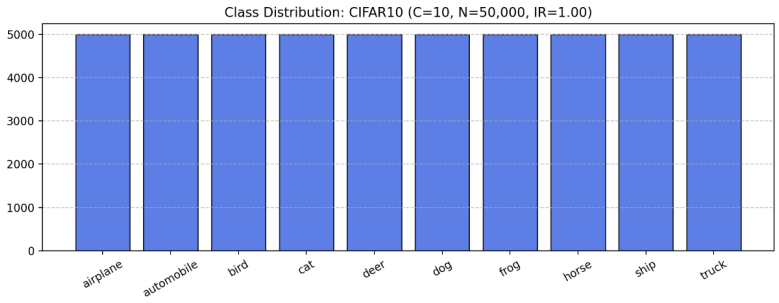
AlexNet-64 + BN on CIFAR-10, 10 epochs, no augmentation, three seeds (sample std, ddof = 1).

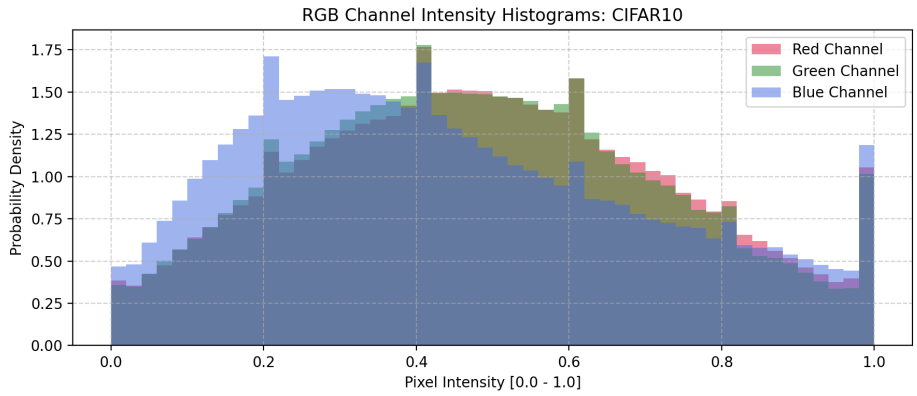
Seed	Train acc (%)	Val acc (%)	Val Macro F1 (%)	Δ
42	84.86	79.93	80.06	4.93
123	85.10	78.72	78.25	6.38
999	85.53	79.04	79.32	6.49
Mean ± Std	85.16 ± 0.34	79.23 ± 0.63	79.21 ± 0.91	5.93 ± 0.87

Ablation summary plot



Pre-lab outputs: demo_transforms.py and EDA





4. Technical analysis and discussion

From LeNet to AlexNet

Switching from Tanh to ReLU and from average to overlapping max pooling raised LeNet's validation accuracy from 53.1% to 64.2% and Macro F1 from 53.1% to 63.8%. The modernized model also converged much faster. Its first-epoch accuracy (52.3%) closely matched the baseline's final result. Tanh saturates for large |x|, where its gradient $1 - \tanh^2(x)$ approaches zero, and these small factors multiply across layers during backpropagation. ReLU has gradient 1 for positive inputs, so the signal survives. Max pooling also keeps the strongest edges, where average pooling blurs it, and the overlap

(K=3 > S=2) will retain more spatial information. Activation, pooling, and padding were changed together, which means we cannot attribute the gain to any single change.

The cost was modest. 1.3x the FLOPs and 32% more parameters were implemented with identical training time on our CPU. In this case, data loading dominates over the arithmetic, so the cost was not significantly affected.

AlexNet-64 reached 73.5% accuracy with no normalization after only 10 epochs, which was about 9 points above the modernized LeNet (10 vs 5 epochs). This reflects the depth, wider channels (64–384 vs 6–16) and much larger receptive field (97 vs 19 pixels) of AlexNet-64 with no normalization. It costs 15x more parameters and about 47x more FLOPs than the modernized LeNet.

Normalization

BatchNorm improved performance as seen in the results. It reached 79.9% accuracy and 80.1% Macro F1, which is about 6.4 points above no normalization and 4.8 points above LRN. It was also ahead from the first epoch (52.6% vs 46.4% and 45.4%), so it converged faster. BN normalizes each channel using mini-batch statistics, which keeps activation scales consistent between layers and keeps gradients well-conditioned. The original explanation is reduced internal covariate shift. More recent evidence attributes much of the benefit to a smoother loss landscape that tolerates larger effective step sizes. Both explanations are consistent with BN's faster and higher convergence.

LRN gave a modest gain, although with only a single seed and 10 epochs this is not a very conclusive finding. This aligns with the finding that LRN does not add much improvement compared with BN.

The profiler reports identical FLOPs for all three variations because it does not count normalization operations. Measured time is therefore the better evidence to conclude from. LRN added about 8% per epoch (25.5 s vs 23.7 s), while BN was the same from no normalization (23.5s). Epoch times were likely limited by CPU data loading rather than GPU arithmetic, so small GPU-side overheads may be hidden.

BN had the fastest and lowest training loss, but its validation curve was the noisiest. A likely cause is a mismatch between BN's running statistics and the rapidly changing weights during evaluation. The no-normalization and LRN curves improved more gradually. BN also showed the largest train–val gap of the three (4.93 vs 1.32 and 3.57). This reflects faster fitting of the training data rather than overfitting, since its validation accuracy was also the highest.

Data Augmentation

Without augmentation, the BN model overfit. The training accuracy climbed to 90.3% while validation accuracy plateaued near 79–81% after epoch 8, and validation loss turned upward after epoch 11. Δ reached 11.72 points. With augmentation, Δ fell to 2.99 points, a reduction of about 75%. Final validation accuracy rose from 78.6% to 81.7%, Macro F1 from 78.3% to 81.3%, and validation loss from 0.710 to 0.522.

The final-epoch comparison is somewhat noisy, since the run that was not augmented dipped at epoch 15. The best-epoch comparison (82.8% vs 81.2%) is the more conservative claim of about 1.6 points. Training accuracy was lower with augmentation, as expected. The model solves a harder problem, so lower training and higher validation accuracy indicates reduced overfitting. Random crops, flips and color jitter preserve the label, so each epoch presents different variants of every image. Because each epoch shows different label-preserving variants of every image, the filters cannot memorize exact pixel positions or colors and must learn features that tolerate shifts, mirroring and lighting changes.

The augmented run had not converged by epoch 15. The validation loss was still falling and training accuracy was still rising, so longer training would likely widen the gain. The time for each epoch rose from about 24s to about 54s with identical FLOPs, because cropping and color jitter run on the CPU, which already limited throughput (about 2,060 to about 930 samples/s). Part of this may reflect Colab CPU variability between sessions.

Tiny ImageNet vs. CIFAR-10

AlexNet learned discriminative features on Tiny ImageNet significantly above the random guess baseline of 1/200 = 0.5%. After epoch 1, the validation accuracy had improved to 6.73%, which is already better than 0.5%. The validation accuracy was the highest after epoch 9 at 29.20%. The training concluded after epoch 10, where AlexNet had a validation accuracy of 28.31%.

Tiny ImageNet had a final validation accuracy of 28.31% and validation Macro F1 of 26.99%, whereas CIFAR-10 had a validation accuracy of 81.65% and Macro F1 of 81.26%. Tiny ImageNet's validation accuracy was much lower than CIFAR-10's. The cause of this could be because Tiny has 200 possible classes and CIFAR only has 10. Since Tiny only has 500 training images per class instead of 5,000, it is much harder to classify. The input photos that Tiny is attempting to classify are 64x64 scenes rather than 32x32 upscaled images, making them even harder to classify with confidence.

The val loss did not diverge or plateau earlier than CIFAR-10. Apart from a small rise at epoch 5, the val loss of Tiny kept falling until epoch 9, only leveled off at epoch 10, and never diverged (Figure X). The train accuracy stayed below the validation accuracy the whole run, with the train accuracy being 27.14% and the validation accuracy being 28.31%, meaning that this model is underfitting and would improve with more epochs. CIFAR-10's train accuracy ended above its validation accuracy (84.64% vs 81.65%), so the CIFAR-10 model was closer to fitting its data.

Model complexity and the PyTorch Zoo baseline

Zoo got 65.24% accuracy and 65.05% F1 on CIFAR-10, compared to 79.93% and 80.06% for AlexNet-64 + BN, despite Zoo having 57.0 million parameters compared to the 6.6 million of AlexNet-64 + BN. Zoo performed worse than all three AlexNet-64 variants. It is not overfitting: the training accuracy was 67.50% and the validation accuracy was 65.24%, a gap of only 2.26 points, meaning the model was underfitting. A reason why Zoo performed worse could be that it was designed for 224x224 images. On a 64x64 input the feature map shrinks to 1x1, so most spatial detail is lost before the FC layers (see Table XX). As stated in the normalization section, BatchNorm added about 6 points to AlexNet-64. Since Zoo does not have BatchNorm, it likely misses out on a similar gain. On Tiny ImageNet, Zoo reached only 11.03%, well below AlexNet-64 + BN's 30.15% under the same settings.

This shows that the complexity of a model doesn't predict accuracy or speed. Zoo has fewer FLOPs (188M vs 438M) because its stride-4 first layer shrinks the image early. However, it is slower, with 1.91 ms vs 1.48 ms latency. Its 57M parameters likely have to be read from memory on every image, so even though there are fewer FLOPs, the extra parameters cost time. More parameters or FLOPs did not lead to better accuracy; the AlexNet-64 design fits smaller images better.

Extended ablation and seed variability

Lowering the learning rate for AlexNet-64 + BN from 1e-3 to 3e-4 raised validation accuracy a little, from 79.93% to 81.02%. The val curve was also smoother, with no big dip at epoch 6, since the steps are smaller and more stable. The epoch time was unchanged. However, it fit the training data harder: train accuracy rose to 89.07%, so the gap between train and val accuracy grew from 4.93 to 8.05 points. Reducing the batch size from 128 to 64 lowered accuracy slightly to 79.17%. With smaller batches there were twice as many updates per epoch (781 instead of 390), but no improvement, and the gap between train and val accuracy stayed about the same (4.86). Each epoch was about 7% slower, increasing from 23.5 s to 25.0 s, because small batches use the GPU less efficiently.

To measure seed variability, the same model was trained 3 times with random seeds 42, 123 and 999. The results were not identical (79.93%, 78.72% and 79.04%), giving a mean of 79.23 ± 0.63%. Random starting weights alone shift the accuracy by about 0.6 points, so a small enough change in results may be something we can ignore. Batch Normalization vs Local Response Normalization (+4.78 points) and Batch Normalization vs no normalization (+6.41 points) are both much larger than the seed-to-seed variation, so these improvements are reliable and not due to chance. Local Response Normalization vs no normalization (+1.63 points) is likely a real but small gain, and the lower learning rate (+1.09 points) is suggestive but not conclusive. The smaller batch size (−0.76 points) is within the seed-to-seed variation, so it had no measurable effect. Seed 42 also gave the best of the three results, so our single-seed baseline is slightly optimistic. However, this was only tested with 3 seeds, so the std value would be more accurate with more seed samples.

Limitations

Most results come from a single seed (42). Between epochs, the validation variations of 2–3 points mean small differences and are not definitive.

The normalization study used 10 epochs and the augmentation study 15, so the two are not directly comparable.

The epoch times reflect a CPU used for data loading and varied across hardware (CPU for LeNet, T4 for AlexNet), so they should not be compared.

The profiler does not count normalization operations, so FLOPs does not show the true cost of LRN and BN.